Tema 6.2: Tareas en Background

Pablo Gordillo (pabgordi@ucm.es)

Departamento de Sistemas Informáticos y Computación

Facultad de Informática



(Basado en el material creado por Puri Arenas, Jesús Correas y Samir Genaim)

Qué es un Worker

- Un objeto (o una clase estática) que mantiene una o más hebras, al que podemos enviar tareas (p. ej. objetos Runnable) para ejecutarlos de forma asíncrona en sus hebras. Puede ser que sea para un propósito muy particular como el "event-dispatch thread" de Swing, SwingWorkers, etc. o para un propósito general.
- Lo importante es que estas hebras nunca "mueren", a menos que paremos el Worker, por lo que evitamos la creación de una nueva hebra para cada tarea que queramos ejecutar de forma asíncrona.

Event Dispatch Thread

- En Java existe el **hilo de despachar eventos (Event Dispatch Thread)** es la hebra en la que se ejecutan todos los eventos de **Swing** (teclado, ratón, etc.).
- Cualquier acción que afecte a **Swing**, debe hacerse en dicha hebra.
- Además la respuesta a los eventos se ejecuta de forma secuencial. Hasta que no acaba la respuesta a un evento, no se procesa la siguiente.
- Por tanto, si la respuesta a un evento tarda mucho, debemos crear otra hebra donde ejecutarla. Pero los resultados obtenidos que hagan referencia a **Swing** deben devolverse a la hebra de **Swing** usando **SwingUtilities.invokeLater()**.
- Desde Java 6, existe la posibilidad de usar la clase **SwingWorker**, que hace todo ese trabajo de forma automática.

SwingWorker

- Para implementar un **SwingWorker**, debemos crear una clase (o implementar una clase anónima) que extienda a **SwingWorker<T,V>**. **V** son los datos calculados y **T** lo que devuelve **doInBackground**. En caso de que no devolvamos o calculemos nada especificaremos **Void** como tipo.
- Esto obligará a implementar el método:

  protected T doInBackground() throws Exception ...

  encargado de realizar los cómputos. **SwingWorker** se encargará de forma automática de que el método **doInBackgroun()** y **done()** se ejecuten en los hilos adecuados. EL primero en un hilo propio para él y el segundo en el hilo de eventos.
- Cuando el método **doInBackground** termina, entonces invoca al método **done()**.

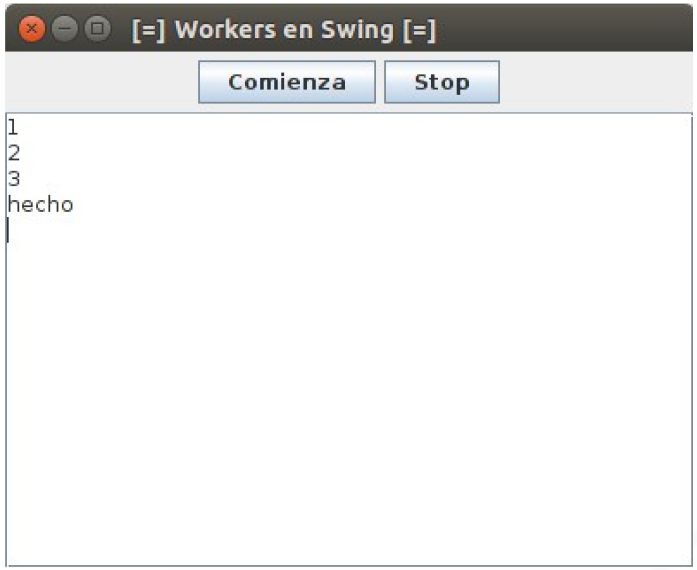
SwingWorker

- Durante la ejecución de **doInBackground()** se puede mostrar el progreso de la tarea en la hebra de Swing.
- Esto se hace en dos pasos:
  1. Mediante una llamada al método **publish(V dato)** (**V** es el segundo parámetro del genérico) desde **doInBackground()**.
  2. El dato en la llamada a **publish** se puede usar para actualizar los componentes **Swing** definiendo el método **process**:
     protected void process(List<V> datos){...}, que recibe una lista con todos los datos generados hasta el momento.
- El método **process** se ejecuta directamente en la hebra de **Swing**. Esto permite comunicar fácilmente resultados intermedios durante el proceso ejecutado en background.

SwingWorker

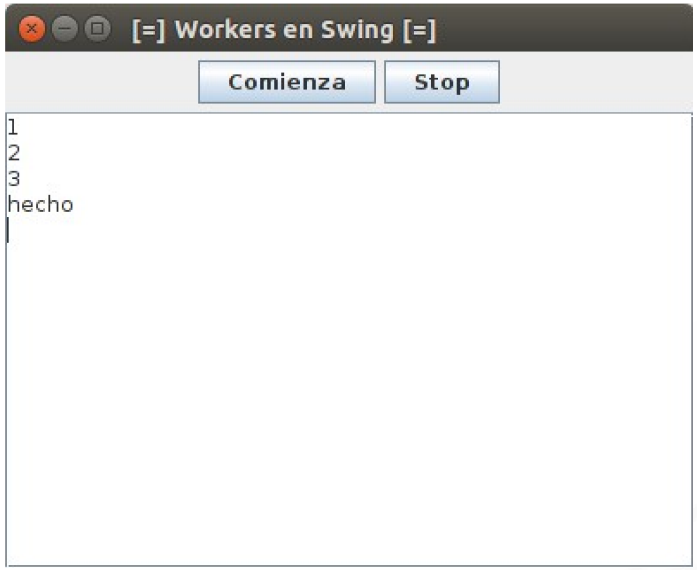
- Se puede ejecutar la ejecución de una tarea en background con el método **cancel(boolean)**
- Como en el caso de **interrupt()**, la tarea debe terminarse a si misma (se puede comprobar si ha sido cancelada con **isCancelled()**).
- Opcionalmente se puede añadir la definición de un método **done()** para ejecutar código cuando ha terminado **doInBackground**.
  - **done()** se ejecuta en la hebra de Swing.
  - Se puede consultar el resultado devuelto por **doInBackground** realizando una llamada el método **public T get()**.
  - Si **doInBackground** no ha terminado, **get()** bloquea la ejecución hasta que termine.

Ejemplo



Comienza:escribe el valor de un contador que se va incrementando
Stop: para la ejecución

Ejemplo



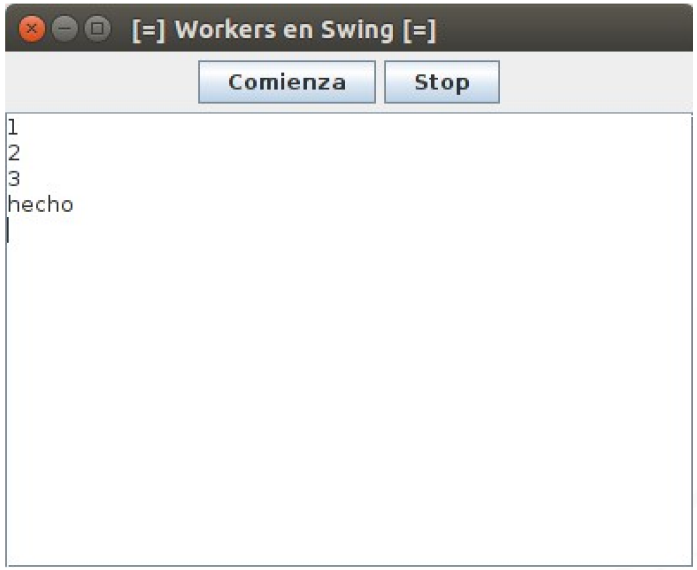

```
public class Worker extends SwingWorker<Void,Integer> {

  @Override
  protected void doInBackground() throws Exception {
    while (! Thread.interrupted ()) {
      try {
        Thread.sleep(500);
        counter++;
        publish (counter); //Manda a Swing el valor
      }
      catch (InterruptedException e) {
        Thread.currentThread().interrupt ();
      }
    }
    return null ;
  }

  ...
```

Comienza:escribe el valor de un contador que se va incrementando
Stop: para la ejecución

Ejemplo



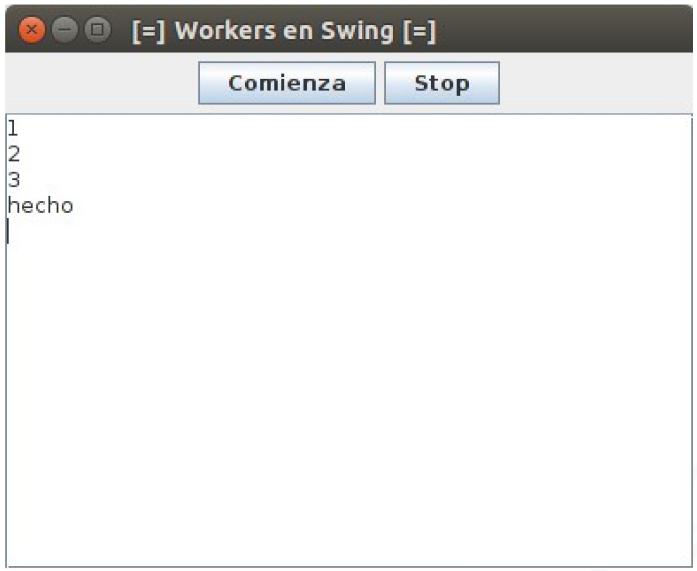
Comienza: escribe el valor de un contador que se va incrementando
Stop: para la ejecución

```
public class Worker extends SwingWorker<Void,Integer> {

  @Override
  protected void doInBackground() throws Exception {
    while (! Thread.interrupted ()) {
      try {
        Thread.sleep(500);
        counter++;
        publish (counter);
      }
      catch (InterruptedException e) {
        Thread.currentThread().interrupt ();
      }
    }
    return null ;
  }

  // process se ejecuta en la hebra de Swing
  @Override
  protected void process(List <Integer> chunks) {
    for (Integer x : chunks) {textArea.append(x + "\n");}
    //textArea lo recibe comoparámetro en el constructor o
      //Worker es clase interna de nuestra ventana
  } // chunks lleva la lista de enteros calculados
```

Ejemplo



Comienza:escribe el valor de un contador que se va incrementando
Stop: para la ejecución

```
public class Worker extends SwingWorker<Void,Integer> {

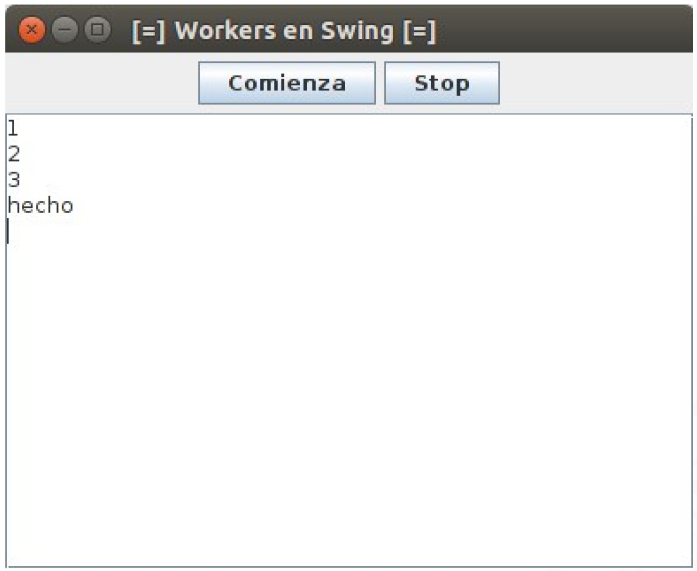
  @Override
  protected void doInBackground() throws Exception {
    while (! Thread.interrupted ()) {
      try {
        Thread.sleep(500);
        counter++;
        publish (counter);
      }
      catch (InterruptedException e) {
        Thread.currentThread().interrupt ();
      }
    }
    return null ;
  }

  @Override
  protected void process(List <Integer> chunks) {
    for (Integer x : chunks) {textArea.append(x + "\n");}

  }

  @Override
  protected void done() {textArea.append("hecho"+"\n");}
} //done se ejecuta en Swing cuando acaba doInBack
```

Ejemplo



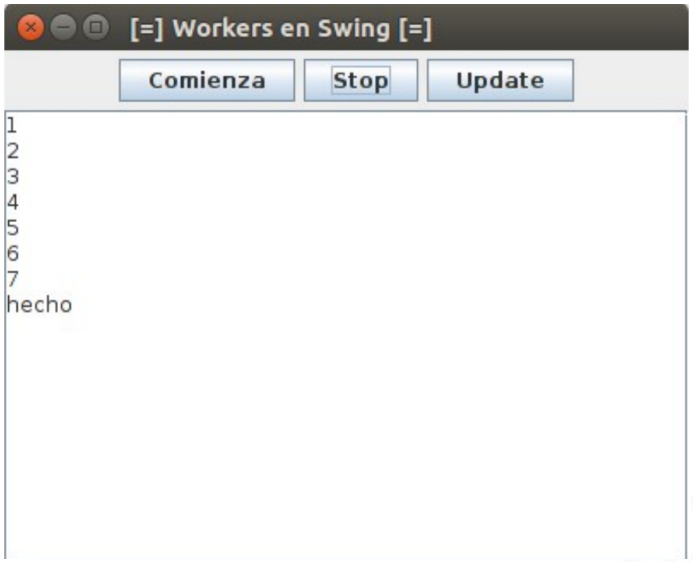
Comienza: escribe el valor de un contador que se va incrementando
Stop: para la ejecución

```
public class Ventana extends JFrame {
  private JTextArea textArea;
  private int counter;
  private SwingWorker<Void,Integer> worker;

  public Ventana() {
    ...
    JButton startButton = new JButton("Comienza");
    startButton.addActionListener(new ActionListener() {

      @Override
      public void actionPerformed(ActionEvent arg0) {
        if (worker==null) {
          worker = new Worker();
          worker.execute();
        } }
    });
    JButton stopButton = new JButton("Stop");
    stopButton.addActionListener(new ActionListener() {
      @Override
      public void actionPerformed(ActionEvent e) {
        if (worker!=null) { worker.cancel(true); worker = null;}
      } // interrumple doInBackground en el worker
    });
    textArea = new JTextArea();
    ...
  }
  ...
```

Otro Ejemplo



El botón update pone el contador a 100

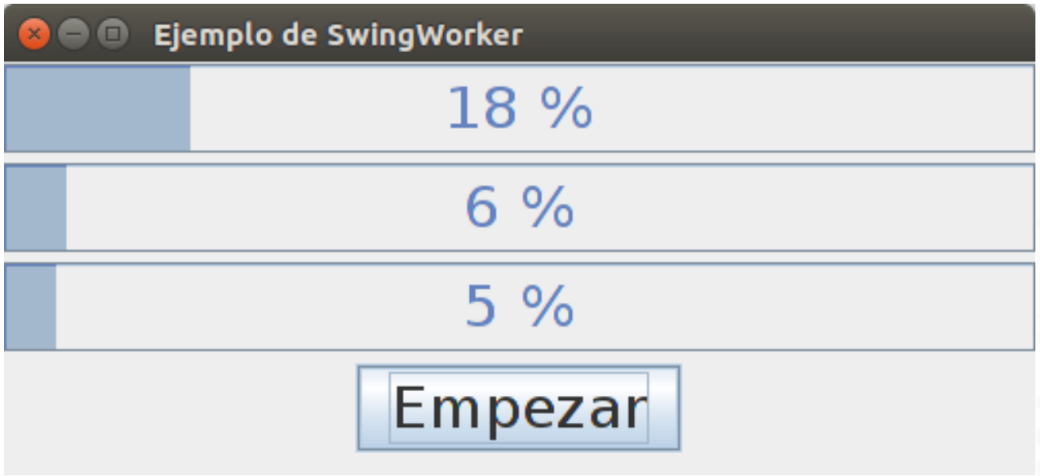
```
updateButton.addActionListener(new ActionListener() {
  @Override
  public void actionPerformed(ActionEvent e) {
    try {
      Thread.sleep(5000);
      counter = 100;
    }
    catch (InterruptedException ex) { }
  }
});
```

Si pulsamos **"Comienza"** y luego **"Update"**, la actualización del contador se hace de forma incorrecta.

"doInBackgound" incrementa **"counter"** en 1, pero no usa la hebra de Swing. Por lo tanto no se ve afectado por la acción del botón **"Update"**.

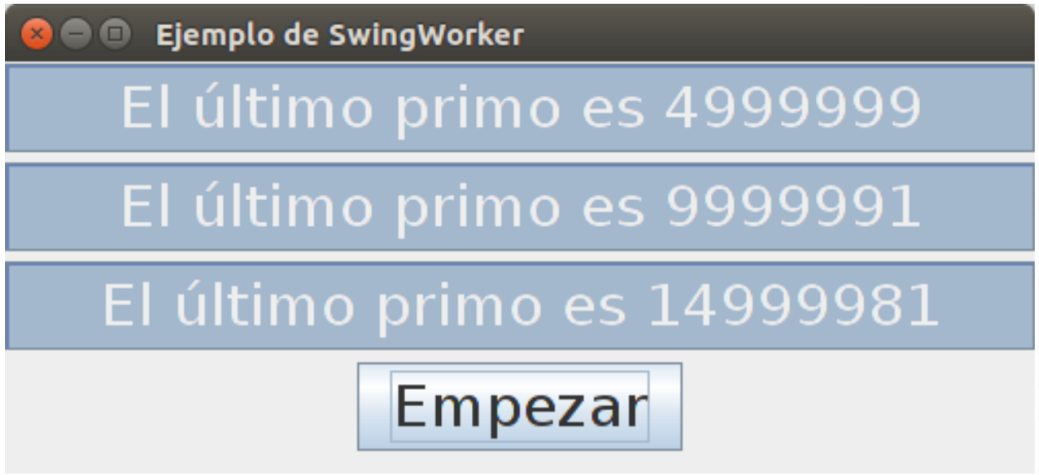
Necesariamente la actualización del contador en **"doInBackground"** debe hacerse en la hebra de Swing para que salgan los resultados esperados.

Calculo de números primos con SwingWorkers



- Calculamos todos los números primos entre 1 y 3*NUM
- Lo hacemos con tres **SwingWorkers**.
  - El primer **Worker** calcula los primos entre 1 y NUM
  - El segundo **Worker** calcula los primos entre NUM+1 y 2*NUM
  - El tercer **Worker** calcula los primos entre 2*NUM+1 y 3*NUM
  - Mostramos el progreso de los **Workers** con barras de progreso

Calculo de números primos con SwingWorkers



- Cuando termina el cálculo de los primos, las barras de progreso muestran el valor del último primo calculado en la hebra.

Calculo de números primos con SwingWorkers

```
public class HebraPrimosWorker extends SwingWorker<Integer,Integer> {

  private int desde; // buscamos los primos entre [desde,hasta]
  private int hasta;
  private JProgressBar pb; // damos valor a la barra de progreso

  public HebraPrimosWorker(JProgressBar pb, int desde, int hasta) {
    this.pb = pb;
    this.desde = desde;
    this.hasta = hasta;
  }

  @Override
  protected Integer doInBackground() throws Exception {
    int ultimo = 0; // almacena el valor del último primo
    for (int i = desde; i <= hasta; i++) {
      if (esPrimo(i)) ultimo=i;
      publish(100*(i-desde)/(hasta-desde)); // se publica el progreso
    }
    return ultimo; // se publica el progreso
  }
```

Calculo de números primos con SwingWorkers

Cuando se llama a **publish()**, entonces se ejecuta el método **process** en la hebra de **Swing**.

```
@Override
protected void process(List<Integer> porcentajes) {
  // porcentajes lleva una lista de todos los elementos
  // publicados por la hebra. Solo queremos el último,
  // que nos indica el porcentaje.
  int porcentaje = porcentajes.get(porcentajes.size()-1);
  // se va mostrando el porcentaje en la barra de progreso
  pb.setString(Integer.toString(porcentaje)+" %");
  // se avanza la barra de progreso hasta el porcentaje
  pb.setValue(porcentaje);
}
```

Calculo de números primos con SwingWorkers

Cuando la hebra termina se ejecuta el método **done** en **Swing**. En dicho método queremos mostrar en la barra de prograso el valor del último primo.
Además decrementamos el número de hebras activas.

```
    @Override
    public void done() {
      // le pedimos a la hebra el último valor calculado
      Integer ultimo = get();
      pb.setString ("El \'ultimo primo es " + ultimo);
      // decrementamos el nÃºmero de workers funcionando
      EjemploWorkerPrimos.this.hebrasPendientes--;
    }
  }
```

hebrasPendientes es un atributo estático que contabiliza el número de workers global

La ventana principal

```
public class EjemploWorkerPrimos extends JFrame implements ActionListener {

  public final static int NUM = 5000000;
  private JProgressBar barraProgreso1, barraProgreso2, barraProgreso3;
  private JButton botonStart;
  private HebraPrimosWorker hebra1, hebra2, hebra3;
  private volatile int hebrasPendientes;

  public EjemploWorkerPrimos() {
    super("Ejemplo de SwingWorker");
    ...
    JPanel panelBarras = this.creaPanelBarras(); // panel barras de progreso
    mainPanel.add(panelBarras);
    JPanel panelBoton = this.creaPanelBoton(); // panel botón Start
    mainPanel.add(panelBoton);
    ...
  }
```

La ventana principal

```
private JPanel creaPanelBarras() {
  JPanel panelBarras = new JPanel(new GridLayout(3,1,5,5));
  barraProgreso1 = this.creaBarraProgreso();
  barraProgreso2 = this.creaBarraProgreso();
  barraProgreso3 = this.creaBarraProgreso();
  panelBarras.add(barraProgreso1);
  panelBarras.add(barraProgreso2);
  panelBarras.add(barraProgreso3);
  return panelBarras;
}

public JProgressBar creaBarraProgreso(){
  JProgressBar barraProgreso = new JProgressBar(0,100);
  barraProgreso.setFont(new Font("Dialog",Font.PLAIN,28));
  barraProgreso.setStringPainted (true);
  return barraProgreso;
}
```

La ventana principal

```
@Override
public void actionPerformed(ActionEvent e) {
  if (hebrasPendientes == 0) {
    hebrasPendientes = 3;
    hebra1 = new HebraPrimosWorker(barraProgreso1,1,NUM);
    hebra2 = new HebraPrimosWorker(barraProgreso2,NUM+1,2*NUM);
    hebra3 = new HebraPrimosWorker(barraProgreso3,2*NUM+1,3*NUM);
    hebra1.execute();
    hebra2.execute();
    hebra3.execute();
  }
  else {
    JoptionPane.showMessageDialog(this, "Todavia hay workers haciendo
    cosas", "Oops!", JOptionPane.INFORMATION_MESSAGE);
  }
}
```